Author: Micheal Lanham

SECTION 1

Three Loop Layers (Inner SPAL, Task Loop, Meta Loop)

- **Layer 1 — inner SPAL:** The agent cycles sense, plan, act, and learn until a goal or termination condition is met; the model chooses tools, judges progress, and decides when to stop within boundaries the developer sets.
- **Core elements:** Every loop variant rests on goal, plan, internal or external state, and an explicit termination decision—the difference between convergence and runaway or premature exit.
- **Layer 2 — task loop:** Long-horizon work externalizes plan and state in code so context does not overwhelm the model and iterations can persist across many short internal cycles.
- **Layer 3 — meta loop:** An agent rather than fixed code controls decide, state, and plan, enabling adaptive replanning; subtype depends on whether one orchestrator delegates or peers collaborate.
- **Chapter focus:** After establishing SPAL, the chapter emphasizes layer-2 task loops and layer-3 orchestration versus collaboration patterns for complex goals.

SECTION 2

Deep Research Task Loop

- **External state and plan:** Typed models such as `ResearchState` and `ResearchPlan` track findings, sources, follow-up questions, subtopics, and iteration counts, serialized via `to_context` to manage window usage.
- **Tools and MCP:** Search and other capabilities attach through MCP servers with proper async lifecycle so tools are available only for the loop's duration.
- **Iteration output:** Strongly typed `ResearchIteration` fields—summary, sources, follow-ups, goal_satisfied, confidence, plan updates—let programmatic controllers parse each cycle without fragile free-text parsing.
- **Termination gate:** Layer hard iteration and budget limits with goal satisfaction, quality thresholds, and stagnation detection when consecutive summaries stop adding new information.
- **Loop controller:** Code pops follow-up questions, applies plan updates, truncates what the agent sees when needed, and exits when `should_continue` is false.
- **Synthesis:** A separate synthesis agent compiles all findings into a structured report after exploration ends, preserving separation between discovery and final narrative.
- **When to loop vs batch:** Use loops for iterative discovery, refinement, conditional branching, and multisource aggregation; skip loops for single-pass classification or simple Q&A. Repetitive task loops drain a defined queue with progress tracking and retries without a separate external plan.

SECTION 3

Multi-Agent Orchestration Loops

- **Orchestrator role:** A meta-level orchestrator agent maintains global plan and state, outputs structured decisions such as delegate, re_plan, or finalize, and never executes search tools directly.
- **Worker specialization:** Research and analysis workers receive narrow assignments and return typed iteration-shaped output the orchestrator merges into shared state.
- **Dynamic replanning:** When worker results reveal gaps, the orchestrator can add or reprioritize subtasks mid-loop—adaptivity a hardcoded layer-2 loop cannot match.
- **Cost tradeoff:** Each orchestrator turn is an extra LLM call; reserve orchestration for goals where decomposition is uncertain or must evolve; use deterministic dispatch when the task list is known upfront.

SECTION 4

Collaborative Agentic Loops

- **Peer pattern:** Multiple agents share CollaborationState and rotate turns under a lightweight manager; there is no single delegator—agents critique and refine collective work.
- **Researcher–critic–synthesizer:** A common triad where one agent gathers evidence, another challenges gaps and contradictions, and a third weaves validated material into coherent output.
- **Consensus termination:** Rounds continue until enough recent contributors agree the goal is met or maximum rounds exhaust, with configurable consensus thresholds.
- **Orchestration vs collaboration:** Orchestration fits independent subtasks and parallel pipelines; collaboration fits interdependent work where adversarial review improves quality—start with orchestration for simpler debugging, add collaboration when output demands rigorous challenge.